Contents

1 Setup

This lab uses the `marketing` dataset from the `datarium` package, which must be installed. Install and import them as necessary.

2 Exploring the data

As usual, look at the structure of the data to understand what you are dealing with. We have 4 continuous numerical variables, each with quite different scales and ranges.

```
str(marketing)
```

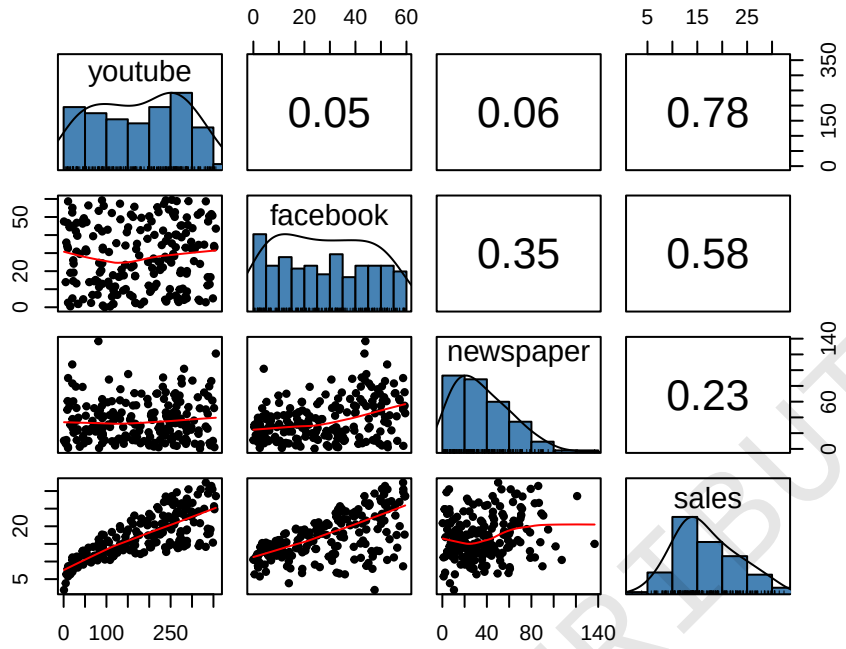
```
## 'data.frame':    200 obs. of  4 variables:
##  $ youtube  : num  276.1 53.4 20.6 181.8 217 ...
##  $ facebook : num  45.4 47.2 55.1 49.6 13 ...
##  $ newspaper: num  83 54.1 83.2 70.2 70.1 ...
##  $ sales    : num  26.5 12.5 11.2 22.2 15.5 ...
```

```
summary(marketing)
```

```
##     youtube          facebook       newspaper          sales      
##  Min.   :  0.84   Min.   : 0.00   Min.   :  0.36   Min.   : 1.92  
##  1st Qu.: 89.25   1st Qu.:11.97   1st Qu.: 15.30   1st Qu.:12.45  
##  Median :179.70   Median :27.48   Median : 30.90   Median :15.48  
##  Mean   :176.45   Mean   :27.92   Mean   : 36.66   Mean   :16.83  
##  3rd Qu.:262.59   3rd Qu.:43.83   3rd Qu.: 54.12   3rd Qu.:20.88  
##  Max.   :355.68   Max.   :59.52   Max.   :136.80   Max.   :32.40
```

We are interested in the `sales` variable.

```
library(psych)
pairs.panels(marketing, ellipses=FALSE, hist.col="steelblue")
```



The scatterplots of each predictor against every other predictor indicates there is no collinearity, so a linear regression will not face any issues.

From the scatterplots of each predictor against `sales`, the outcome variable of interest, `youtube` and `facebook` have a higher correlation and thus more predictive power as compared to `newspaper`. Thus, when improving the model, we should consider these two predictors.

3 Preparing the data

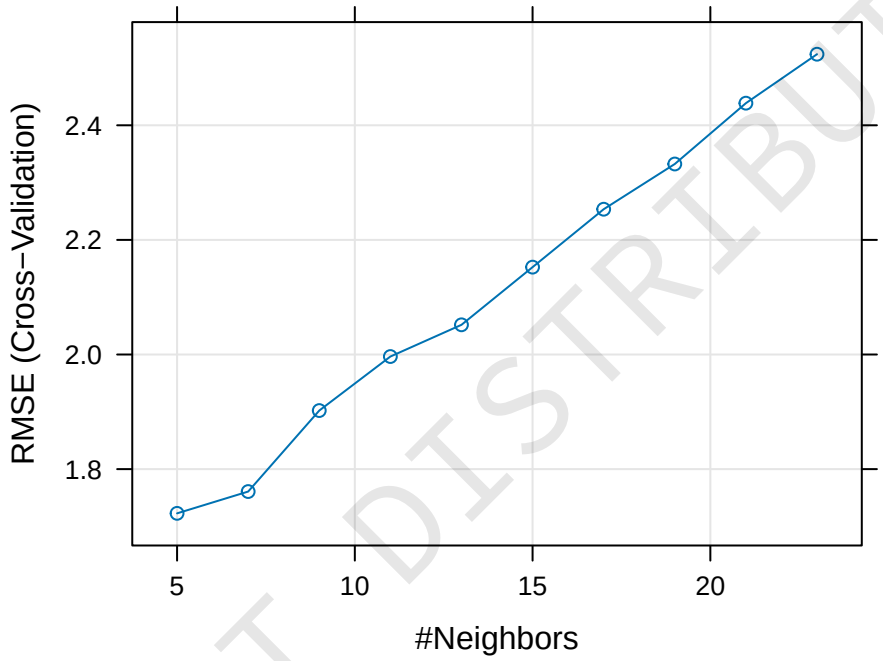
```
set.seed(100)
train.idx <- sample(nrow(marketing), nrow(marketing)*0.8)
train.data <- marketing[train.idx,]
test.data <- marketing[-train.idx,]
```

4 Training the models

4.1 kNN

Our training set has 160 observations, to get the guideline of 40-50 samples in each fold, set `number = 4`. The kNN model achieves a test RMSE of 1.367.

```
library(caret)
set.seed(101)
knn <- train(sales ~ ., data = train.data, method = "knn",
             trControl = trainControl("cv", number=4),
             preProcess = c("center", "scale"),
             tuneLength = 10)
plot(knn)
```




```
yhat <- predict(knn, test.data)
y <- test.data$sales
rmse.knn <- RMSE(yhat, y)
rmse.knn
```

```
## [1] 1.366957
```

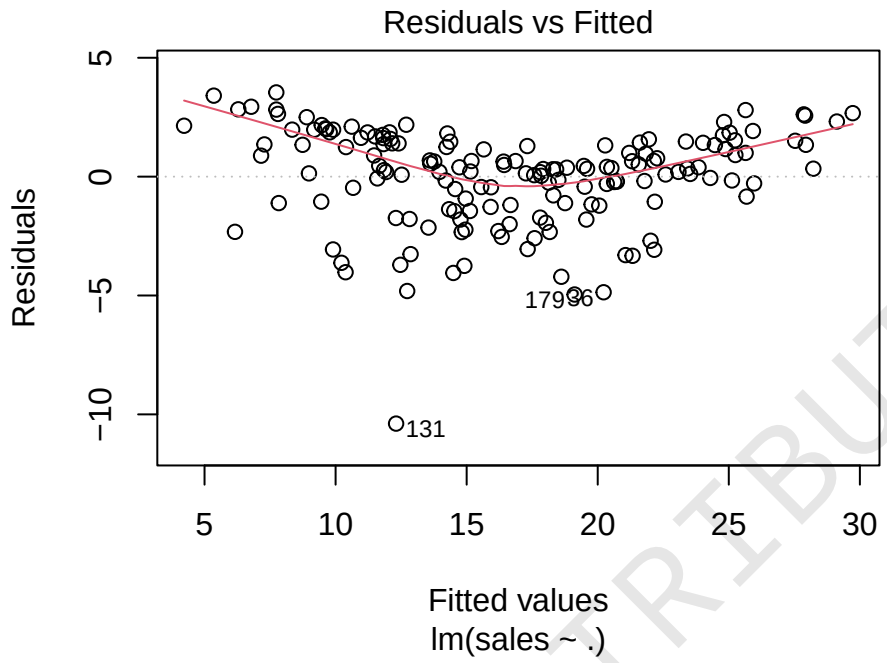
4.2 Linear regression

Fit the model, and look at its summary and residual plot. From the residual plot, we can see a quadratic trend, indicating we could consider adding quadratic terms. We will also track the RMSE so that we can compare it to other models. This simple linear regression model achieves a test RMSE of 1.954.

```r
lm.slr <- lm(sales ~ ., train.data)
summary(lm.slr)
```

```
##
## Call:
## lm(formula = sales ~ ., data = train.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.3885  -1.1128   0.3308   1.4250   3.5426
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 3.292962   0.427857   7.696 1.49e-12 ***
## youtube     0.045773   0.001566  29.235  < 2e-16 ***
## facebook    0.188077   0.010113  18.597  < 2e-16 ***
## newspaper   0.003798   0.006958   0.546    0.586
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.046 on 156 degrees of freedom
## Multiple R-squared:  0.8925, Adjusted R-squared:  0.8905
## F-statistic: 431.9 on 3 and 156 DF,  p-value: < 2.2e-16
```

```r
plot(lm.slr, which=1)
```




```
yhat <- predict(lm.slr, test.data)
rmse.slr <- RMSE(yhat, y)
rmse.slr
```

```
## [1] 1.95369
```

4.3 Improving the linear regression model

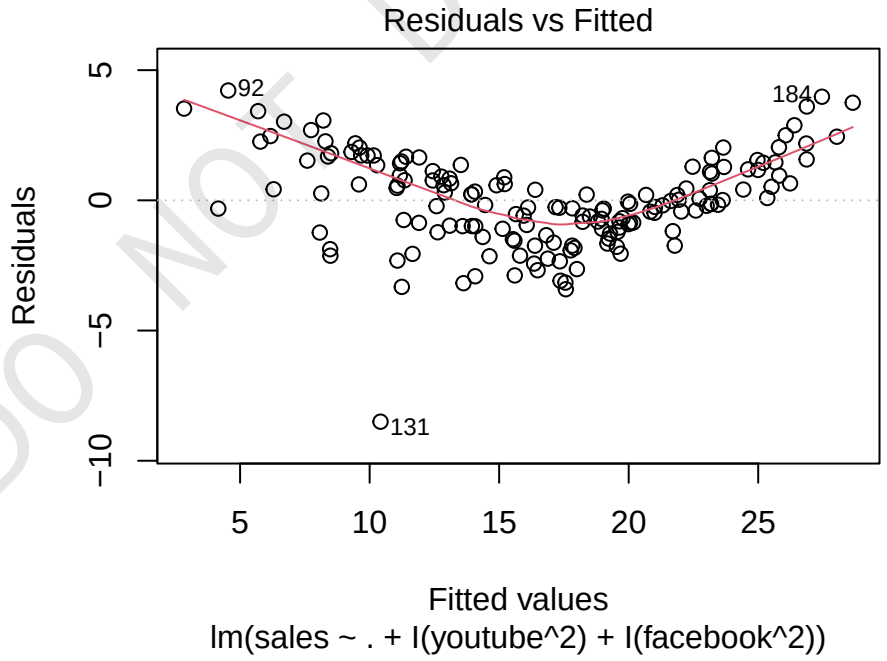
Since the residual plot suggests a quadratic pattern, we attempt to improve the model by adding quadratic terms of the predictors with higher predictive power, `youtube` and `facebook`. However, the residual pattern remains largely similar with the quadratic pattern, and the test RMSE also is much unchanged at 1.955.

```
lm.quadratic <- lm(sales ~ . + I(youtube^2) + I(facebook^2),
↪  data=train.data)
summary(lm.quadratic)
```

```
##
## Call:
## lm(formula = sales ~ . + I(youtube^2) + I(facebook^2), data = train.data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
```

```
## -8.5001 -1.0560 -0.0925  1.2134  4.2185
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)     1.303e+00  5.937e-01   2.195   0.0297 *
## youtube         8.208e-02  5.516e-03  14.881  < 2e-16 ***
## facebook        1.666e-01  3.220e-02   5.174 7.04e-07 ***
## newspaper       6.170e-03  6.158e-03   1.002   0.3179
## I(youtube^2)   -1.054e-04  1.548e-05  -6.809 2.07e-10 ***
## I(facebook^2)   4.726e-04  5.438e-04   0.869   0.3861
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.799 on 154 degrees of freedom
## Multiple R-squared:  0.918,  Adjusted R-squared:  0.9153
## F-statistic: 344.7 on 5 and 154 DF,  p-value: < 2.2e-16
```

```
plot(lm.quadratic, which=1)
```




```
yhat <- predict(lm.quadratic, test.data)
rmse.quadratic <- RMSE(yhat, y)
rmse.quadratic
```

```
## [1] 1.954943
```

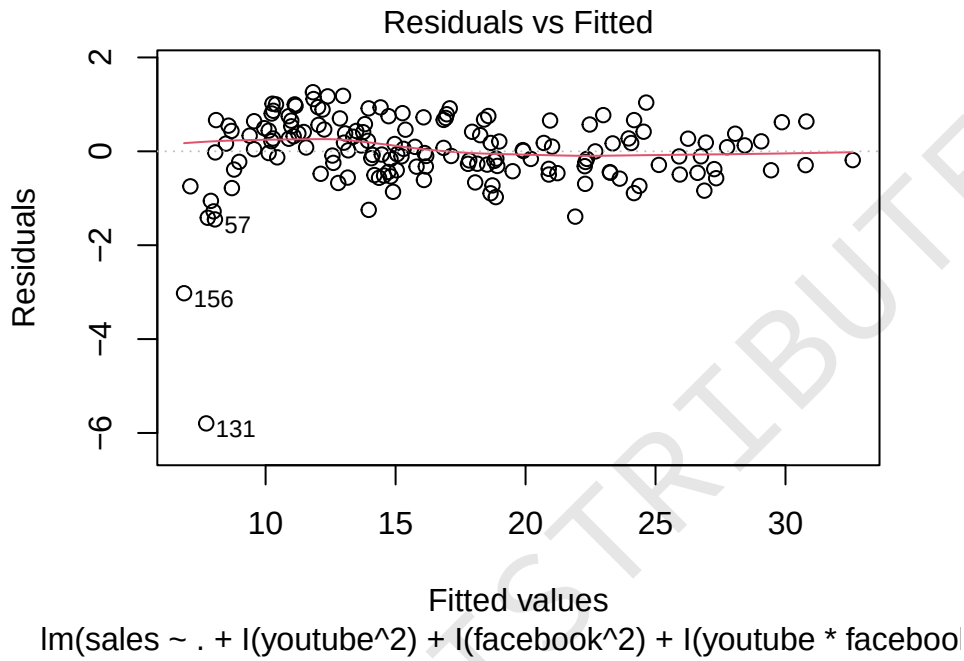
Another possibility is to consider the **interaction** between the terms. The term `youtube*facebook` considers the interaction between these two predictors, and allows for modelling the interaction between these two predictors, e.g. a higher `youtube` and a lower `facebook`. A real world interpretation may be that when advertising, one may wish to target one platform more than another, depending on their target audience.

We add this term in the same way for the quadratic terms, with `I()`. The R^2 increases to 98%, the residual plot is much improved, and the test RMSE achieved is 0.513, a big improvement from 1.954.

```
lm.interaction <- lm(sales ~ . + I(youtube^2) + I(facebook^2) +
↪  I(youtube*facebook), data=train.data)
summary(lm.interaction)
```

```
##
## Call:
## lm(formula = sales ~ . + I(youtube^2) + I(facebook^2) + I(youtube *
##     facebook), data = train.data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -5.7959 -0.3815  0.0489  0.4718  1.2595
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             6.163e+00  3.270e-01  18.848   <2e-16 ***
## youtube                 5.129e-02  2.738e-03  18.734   <2e-16 ***
## facebook                2.378e-02  1.538e-02   1.546   0.1241
## newspaper               4.579e-03  2.733e-03   1.676   0.0959 .
## I(youtube^2)           -9.325e-05  6.887e-06 -13.540   <2e-16 ***
## I(facebook^2)           1.309e-04  2.417e-04   0.542   0.5888
## I(youtube * facebook)   9.027e-04  3.599e-05  25.084   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7983 on 153 degrees of freedom
## Multiple R-squared:  0.984,  Adjusted R-squared:  0.9833
## F-statistic:  1564 on 6 and 153 DF,  p-value: < 2.2e-16
```

```
plot(lm.interaction, which=1)
```




```
yhat <- predict(lm.interaction, test.data)
rmse.interaction <- RMSE(yhat, y)
rmse.interaction
```

```
## [1] 0.5134645
```

5 Comparison of kNN and LR models

Looking at the test RMSE of all the models, we can see that the LR model with the interaction term performed the best. The simple linear regression model, and the regression with quadratic terms performed the worst. The better performance of the kNN model compared to the LR models without interaction is some evidence that the relation between `sales` and its predictors is nonlinear.

```
cbind(rmse.knn, rmse.slr, rmse.quadratic, rmse.interaction)
```

```
##      rmse.knn rmse.slr rmse.quadratic rmse.interaction
## [1,] 1.366957  1.95369       1.954943        0.5134645
```

The linear regression with interaction outperforms every other model in terms of the RMSE, and so we will choose to use it.

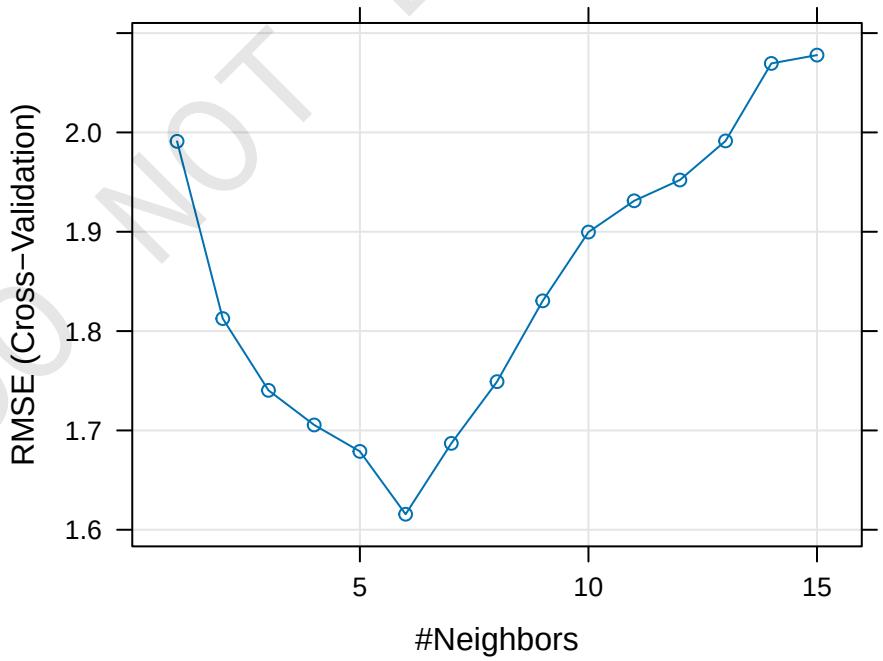
6 Extra: kNN: custom values of k

In the above kNN RMSE plot from the training phase, the RMSE immediately increases. This is troubling, as it indicates the 'best' k value may have occurred at some value *smaller than 5*. To specify the exact values of k that we wish to test, we may replace the `tuneLength` parameter with `tuneGrid`.

`tuneGrid` accepts a data frame of the parameters that should be tested. For kNN, we only have one parameter, k, so we pass it a data frame with a single column k, and the values we wish for it to test - in this case, from $k = 1$ to $k = 15$. The training phase will thus test 15 values of k in the same cross-validation process.

This process results in a different selection of k, as the training RMSE was at a minimum at $k = 6$. The evaluation on the test set is thus slightly different than the original model with $k = 5$ - although the two models are still comparable.

```
set.seed(101)
knn.all <- train(sales ~ ., data = train.data, method = "knn",
                 trControl = trainControl("cv", number=5),
                 preProcess = c("center", "scale"),
                 tuneGrid = data.frame(k=seq(1,15)))
plot(knn.all)
```




```
yhat <- predict(knn.all, test.data)
rmse.knn.all <- RMSE(yhat, y)
cbind(rmse.knn, rmse.knn.all)
```

```
##      rmse.knn rmse.knn.all
## [1,] 1.366957     1.487572
```